

Predykcja Cukrzycy za Pomocą Modeli Machine Learning

Iciek Michał
Bartłomiej Burnus



Charakterystyka Zbioru Danych





W naszym projekcie wykorzystaliśmy dane pochodzące z zbioru **diabetes.csv**, który był analizowany również na innym przedmiocie. Zbiór danych zawiera informacje na temat czynników zdrowotnych i ich związku z występowaniem cukrzycy u pacjentów.

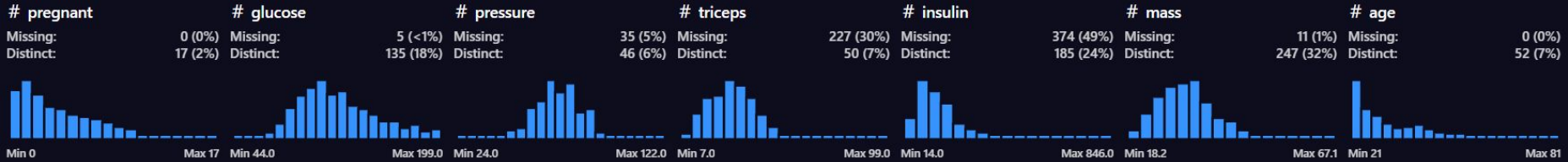
Zbiór danych składa się z 8 kolumn:

- **pregnant** – liczba ciąż (typ: *int*)
- **glucose** – stężenie glukozy w osoczu (typ: *float*)
- **pressure** – ciśnienie rozkurczowe (typ: *float*)
- **triceps** – grubość fałdu skórniego na tricepsie (typ: *float*)
- **mass** – wskaźnik masy ciała (BMI) (typ: *float*)
- **age** – wiek pacjenta (typ: *int*)
- **diabetes** – informacja o występowaniu cukrzycy (typ: *string*), z dwoma unikalnymi wartościami:
 - **neg** – brak cukrzycy
 - **pos** – cukrzyca

Wszystkie kolumny, poza **pregnant**, **age** i **diabetes**, mają wartości typu *float*. Kolumna **pregnant** i **age** przechowuje wartości całkowite, a **diabetes** to ciąg znaków wskazujący na obecność lub brak choroby.

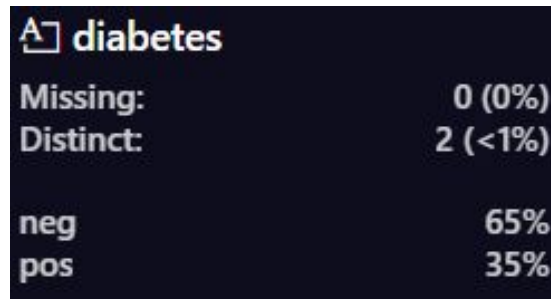
Wstępna Analiza Danych





Zbiór danych został poddany wstępnej analizie, podczas której zidentyfikowano rozkład wartości w poszczególnych kolumnach oraz braki danych:

- pregnant** (liczba ciąż):
 - Braki danych: brak brakujących wartości (0%).
 - Liczba unikalnych wartości: 17 (2%).
- glucose** (stężenie glukozy w osoczu):
 - Braki danych: 5 braków (<1%).
 - Liczba unikalnych wartości: 135 (18%).
- pressure** (ciśnienie rozkurczowe):
 - Braki danych: 35 braków (5%).
 - Liczba unikalnych wartości: 46 (6%).
- triceps** (grubość fałdu skórniego na tricepsie):
 - Braki danych: 227 braków (30%).
 - Liczba unikalnych wartości: 50 (7%).
- insulin** (poziom insuliny):
 - Braki danych: 374 braków (49%).
 - Liczba unikalnych wartości: 185 (24%).
- mass** (BMI):
 - Braki danych: 11 braków (1%).
 - Liczba unikalnych wartości: 247 (32%).
- age** (wiek):
 - Braki danych: brak brakujących wartości (0%).
 - Liczba unikalnych wartości: 52 (7%).
- diabetes** (występowanie cukrzycy):
 - Braki danych: brak brakujących wartości (0%).
 - Liczba unikalnych wartości: 2 (<1%).





Wnioski

- Kolumny **triceps**, **insulin**, oraz **mass** zawierają znaczne braki danych (odpowiednio 30%, 49% i 1%), co wymaga zastosowania odpowiednich metod imputacji lub innego podejścia w celu przetworzenia braków.
- Kolumny mają zróżnicowaną liczbę unikalnych wartości, co może mieć wpływ na modelowanie – warto szczególnie zwrócić uwagę na rozkład cech liczbowych.
- Dane są generalnie kompletne w kolumnach **pregnant**, **glucose**, oraz **age**, co stanowi dobrą podstawę do analizy i modelowania.
- Dane w kolumnie **diabetes** są nieproporcjonalne, co wymaga uwzględnienia nierównowagi klas podczas budowy i oceny modelu (np. przez ważenie klas, oversampling lub wybór odpowiednich metryk, takich jak F1-score).

Wstępna Obróbka Danych





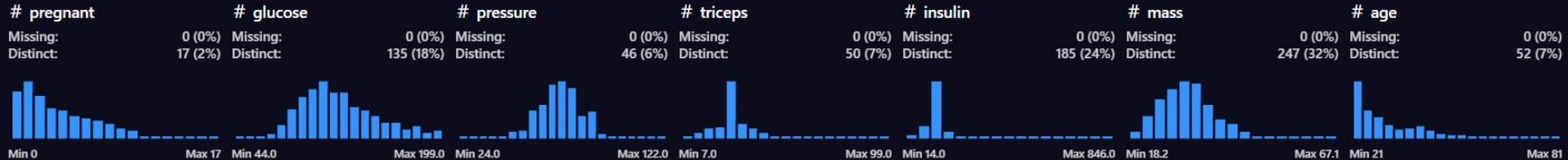
Niekompletne dane

W przypadku brakujących danych zdecydowaliśmy się na ich zastąpienie **medianą**. Mediana, jako miara **odporna na wartości odstające**, pozwala zachować **reprezentatywność danych**. Uznaliśmy, że takie podejście jest lepsze niż usuwanie obserwacji z brakami, ponieważ pozwala na zachowanie jak **największej ilości informacji w zbiorze danych**.

```
columns_with_na = ["glucose", "pressure", "triceps", "insulin", "mass"]
for col in columns_with_na:
    dane[col].fillna(dane[col].median(), inplace=True)
```


Przygotowanie danych do modelu





Wartości w kolumnie *diabetes* zostały zamienione z formatu tekstowego (*neg/pos*) na numeryczny (0/1). Następnie odpowiednie kolumny zostały przypisane do zmiennych:

- **X**: zawiera dane wejściowe (cechy predykcyjne),
- **y**: zawiera wartości docelowe (etykiety, czyli wynik diagnozy cukrzycy).

Dane zostały podzielone na dwa zbiory:

- **zbiór treningowy**: wykorzystywany do uczenia modelu,
- **zbiór testowy**: przeznaczony do oceny skuteczności modelu.

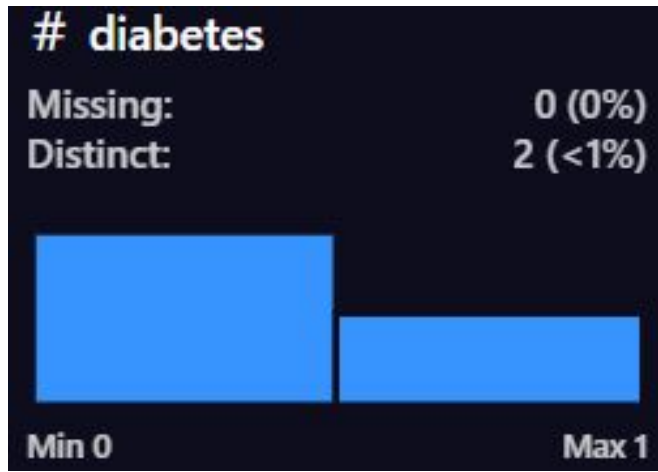
W celu poprawy wydajności modelu dodano syntetyczne dane, co pozwoliło zrównoważyć klasy i zwiększyć skuteczność przewidywań.

```
dane['diabetes'] = dane['diabetes'].replace({'neg': 0, 'pos': 1})
```

```
X = dane.drop(columns=['diabetes'])
y = dane['diabetes']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8, stratify=y, random_state=42)
```

```
smote = SMOTE()
X_train, y_train = smote.fit_resample(X_train, y_train)
```



Model



Architektura modelu:

- Model sekwencyjny zbudowany z **4** warstw w pełni połączonych (*Dense*).
- Wymiary wejściowe: **7 cech predykcyjnych**.
- Liczba neuronów w kolejnych warstwach: **64, 32, 16, 1**.

Funkcje aktywacji:

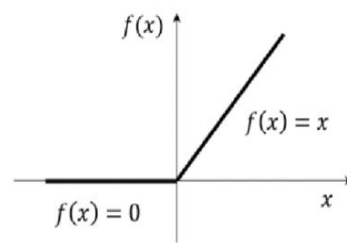
- Warstwy ukryte: **ReLU** oraz **LeakyReLU** (z wartościami $\alpha = 0.3$ i 0.4), co zapewnia elastyczność w modelowaniu nieliniowości.
- Warstwa dropout: Aby zredukować overfitting
- Warstwa wyjściowa: **Sigmoid** – idealna do klasyfikacji binarnej.

Optymalizacja:

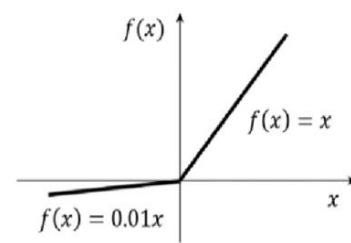
- Algorytm: *Adam* z ustawionym współczynnikiem uczenia (**learning rate=0.001**).
- Funkcja straty: **binary_crossentropy** – używana przy klasyfikacji binarnej.

Ocena modelu:

- Metryki:
 - **Accuracy** – ogólna dokładność modelu.
 - **Precision** i **Recall** – szczególnie istotne przy nierównowadze klas.
 - **AUC** – mierzy jakość klasyfikatora niezależnie od progów.



ReLU activation function



LeakyReLU activation function

```
model = keras.Sequential()
model.add(keras.layers.Dense(64,activation='relu',input_dim=7))
model.add(LeakyReLU(alpha=0.3))
model.add(keras.layers.Dense(32,activation='relu'))
model.add(LeakyReLU(alpha=0.3))
model.add(keras.layers.Dense(16,activation='relu'))
model.add(LeakyReLU(alpha=0.4))
model.add(keras.layers.Dropout(0.3))
model.add(keras.layers.Dense(1,activation='sigmoid'))
```

```
model.compile(Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy', 'Precision', 'Recall', 'AUC'])
```

Early Stopping:

Zastosowano mechanizm wczesnego zatrzymywania treningu, aby uniknąć przeuczenia modelu. Jeśli przez 50 kolejnych epok wynik walidacyjny (val_loss) nie poprawia się, proces treningu zostaje automatycznie przerwany.

Resetowanie wag:

Dzięki funkcji `reset_weights(model)` możliwe jest przywrócenie wag i biasów modelu do ich początkowych wartości. Pozwala to na wielokrotne trenowanie modelu od tych samych ustawień startowych.

Proces trenowania:

Model został skonfigurowany do maksymalnie 1000 epok z następującymi ustawieniami:

- **Wielkość batcha:** 32
- **Shuffle:** Tak, aby dane treningowe były przemieszane przed każdą epoką
- **Walidacja:** Użyto danych testowych do oceny wyników w trakcie treningu

Podsumowanie:

Połączenie wczesnego zatrzymywania treningu z możliwością resetowania wag zapewnia efektywne zarządzanie zasobami. Dzięki temu można wielokrotnie eksperymentować z modelem bez wpływu wcześniejszych trenowań.

```
ES = keras.callbacks.EarlyStopping(  
    monitor = 'val_loss',  
    patience = 50,  
)
```

Tabnine | Edit | Test | Explain | Document

```
def reset_weights(model):  
    for layer in model.layers:  
        if hasattr(layer, 'kernel_initializer') and layer.kernel is not None:  
            layer.kernel.assign(layer.kernel_initializer(tf.shape(layer.kernel)))  
        if hasattr(layer, 'bias_initializer') and layer.bias is not None:  
            layer.bias.assign(layer.bias_initializer(tf.shape(layer.bias)))
```

```
history = model.fit(X_train, y_train, epochs=1000, batch_size=32, shuffle = True, validation_data=[X_test,y_test],callbacks=ES,verbose=1)
```

Metryki modelu i Predykcja

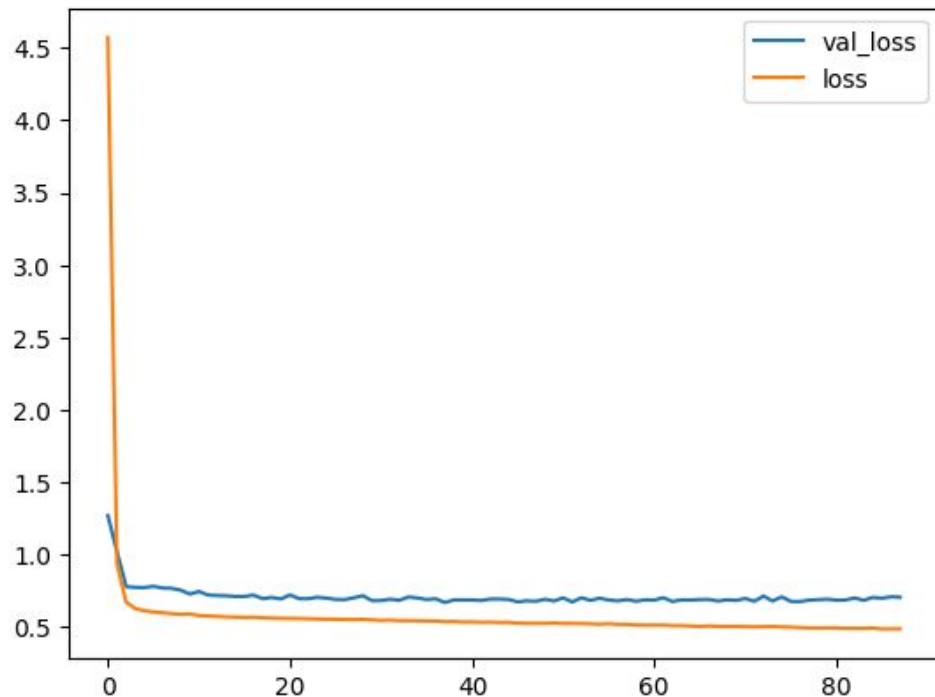




Przebieg Loss i Validation Loss podczas trenowania

Na wykresie przedstawiono przebieg funkcji straty (*loss*) dla zbioru treningowego oraz walidacyjnego w kolejnych epokach.

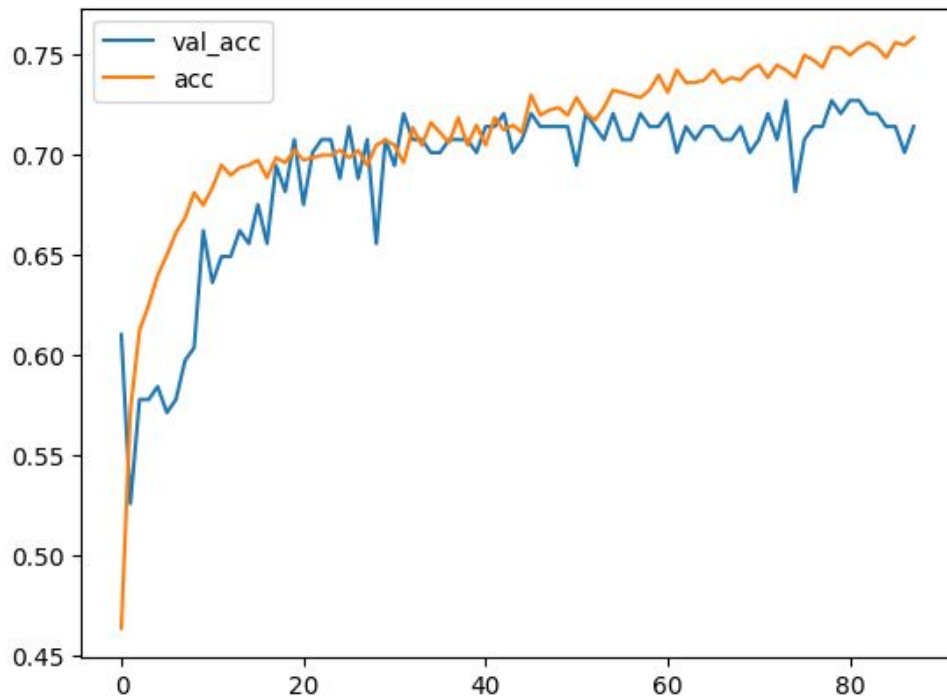
- Początkowo wartość *loss* maleje gwałtownie, co świadczy o szybkim dostosowywaniu modelu do danych.
- Z czasem wartości *loss* stabilizują się na niższym poziomie.
- Różnica między *loss* a *val_loss* wskazuje na możliwość lekkiego przeuczenia (*overfitting*), ponieważ *val_loss* zaczyna wzrastać po pewnej liczbie epok, podczas gdy *loss* na zbiorze treningowym nadal maleje.





Przebieg Accuracy i Validation Accuracy podczas trenowania

- Początkowo obie krzywe wykazują wzrost, co oznacza, że model uczy się poprawnie.
- Dokładność na zbiorze treningowym (acc) rośnie systematycznie i osiąga wyższe wartości niż dokładność walidacyjna.
- Dokładność walidacyjna (val_acc) jest bardziej niestabilna, co wynika z większej zmienności danych walidacyjnych.
- W późniejszych epokach widoczna jest różnica między acc a val_acc, co może sugerować overfitting – model dobrze dopasowuje się do danych treningowych, ale nie poprawia już swojej skuteczności na zbiorze walidacyjnym.





Ocena modelu – Macierz konfuzji i dokładność

Macierz konfuzji:

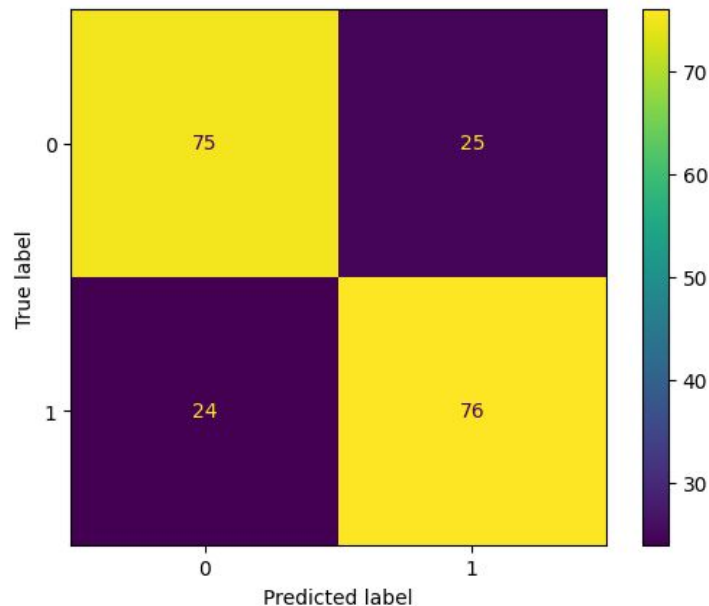
- *True Positives (1,1)*: 76 poprawnych predykcji dla klasy 1.
- *True Negatives (0,0)*: 75 poprawnych predykcji dla klasy 0.
- *False Positives (0,1)*: 25 błędnych predykcji dla klasy 1.
- *False Negatives (1,0)*: 24 błędne predykcje dla klasy 0.

Model wykazuje umiarkowaną skuteczność, zrównoważoną między klasami, ale widoczny jest margines błędu w obu klasach.

Dokładność modelu (accuracy):

- **Wynik**: 75,5%.
- Stosunek poprawnych klasyfikacji do ogólnej liczby przykładów.

Wnioski: Model działa dobrze, ale dalsze ulepszenia, takie jak dostosowanie hiperparametrów lub techniki balansowania danych, mogą zwiększyć jego efektywność.



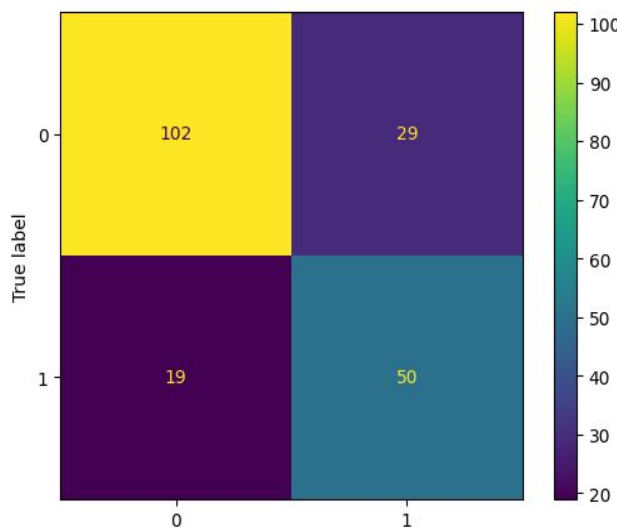
```
accuracy_score(true_class, predykcje_class)
```

✓ 0.0s

0.755

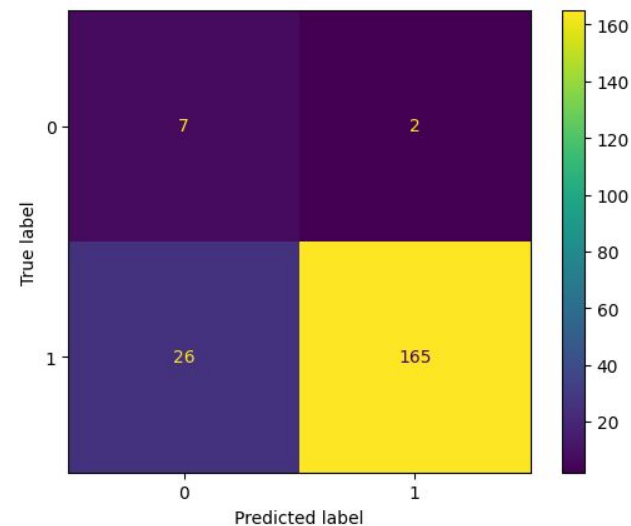
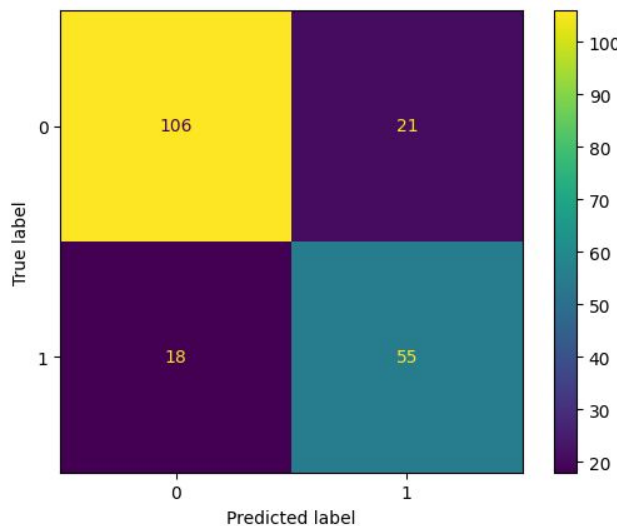
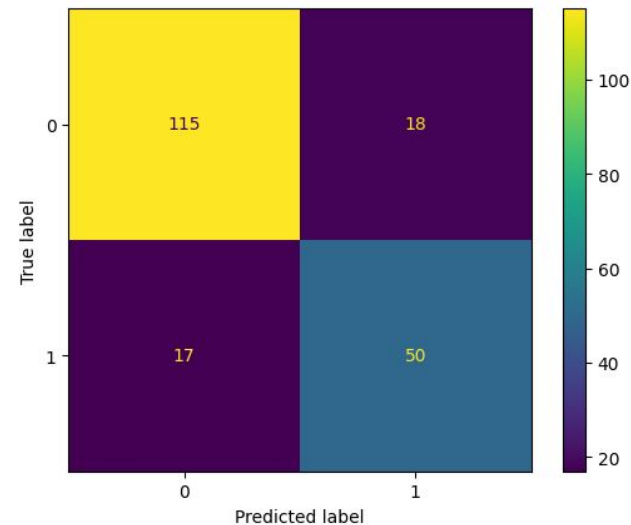
Kowalidacja





Poprawa wydajności modelu jest widoczna z iteracji na iterację. Ostatnia macierz wskazuje na dobrze zoptymalizowany model, który skutecznie klasyfikuje dane wejściowe.

- Wysoka liczba prawidłowych klasyfikacji dla klasy 0 (TN) i klasy 1 (TP).
- Liczba błędnych klasyfikacji (FP i FN) różni się w zależności od iteracji.
- Ostatnia macierz wskazuje na największą poprawność predykcji dzięki optymalizacji modelu.



- **Precyzja (Precision):** Mierzy, jak duży procent przypadków sklasyfikowanych jako pozytywny faktycznie jest pozytywny. W pierwszej iteracji wynosiła 0.63, ale w czwartej iteracji wzrosła aż do 0.99, co wskazuje na znaczne zmniejszenie liczby FP.
- **Czułość (Recall):** Wskaźnik, który pokazuje, ile przypadków rzeczywistej klasy 1 zostało poprawnie wykrytych. Początkowa czułość 0.72 w pierwszej iteracji wzrosła do 0.86 w czwartej iteracji, co oznacza poprawę w redukcji FN.
- **F1-score:** Harmoniczna średnia precyzji i czułości, będąca wskaźnikiem równowagi między tymi dwiema metrykami. Początkowy wynik 0.68 w pierwszej iteracji wzrósł do 0.92 w ostatniej, co świadczy o doskonałej równowadze między precyzją a czułością.
- **Średnia dokładność:** Średnia dokładność modelu z wszystkich iteracji, wynosząca 81,25%, co wskazuje na stabilne i dobre działanie modelu.
- **Odchylenie standardowe:** Wynosi 3,61%, co oznacza niewielkie rozproszenie wyników między iteracjami i świadczy o spójności działania modelu.

Wnioski:

- W każdej iteracji widać poprawę metryk, co jest dowodem na skuteczność procesu optymalizacji.
- Ostateczny wynik (Precyzja – 0.99, Recall – 0.86, F1-score – 0.92) wskazuje na bardzo dobry poziom dopasowania modelu, przy minimalnej liczbie błędów w klasyfikacji.
- Precyzja jest szczególnie wysoka, co oznacza, że model rzadko popełnia błędy typu FP, czyli błędnie klasyfikuje dane jako pozytywne.
- Model wykazuje stabilność w wynikach pomiarów, co potwierdza jego solidność i niezawodność w klasyfikacji.

```
glob_acc = np.mean(solo_accs)
glob_acc_std = np.std(solo_accs)
print(f"global acc: {glob_acc} std: {glob_acc_std}")
```

✓ 0.0s

global acc: 0.8125 std: 0.03614208073700239

```
for idx, metrics in enumerate(all_metrics):
    print(f"Iteration {idx + 1}:")
    print(f" Precision: {metrics['precision']:.2f}")
    print(f" Recall: {metrics['recall']:.2f}")
    print(f" F1-score: {metrics['f1_score']:.2f}")
    print("-" * 40)
```

✓ 0.0s

Iteration 1:

Precision: 0.63

Recall: 0.72

F1-score: 0.68

Iteration 2:

Precision: 0.74

Recall: 0.75

F1-score: 0.74

Iteration 3:

Precision: 0.72

Recall: 0.75

F1-score: 0.74

Iteration 4:

Precision: 0.99

Recall: 0.86

F1-score: 0.92